Assignment 1

Course Code	**UE24CS1203**
Course Title	**Data Structures & Applications**
Faculty Member	**Yashoda M S**
Issue Date	**18/04/2025**
Submission Date	**30/04/2025**

1. Text Editor Functionality Using Stack

Objective:
Create a mini text editor that allows users to input text with the ability to perform **undo** and **redo** operations using **stack data structures**.

Features:

1. **Text Input Area**
   • Users can type and edit text in a simple editor box.
2. **Undo/Redo Functionality**
   • Use two stacks:
   - **Undo Stack:** Stores previous states of the text.
   - **Redo Stack:** Stores undone states for redo functionality.
     • "Undo" button pops from Undo Stack and pushes to Redo Stack.
     • "Redo" button pops from Redo Stack and pushes to Undo Stack.
3. **Display Area**
   • Dynamically updates the text display on every change.
4. **Styling and Layout**
   • Use CSS to format the editor box, undo/redo buttons, and status indicators.

1) **Marks Rubrics:**

Categories	Obtain Marks	Marks
Text Input Area (2 Marks)	Text box is functional and allows smooth input/editing. UI clearly shows current text content.	
Undo/Redo Stack Functionality (3 Marks)	Undo/Redo buttons work correctly using stacks. Each action updates the display area and stacks appropriately.	
Display and Status Area (2 Marks)	Text content updates dynamically on user actions. Status indicators (like "Undo performed") are clear.	
Styling and Layout (2 Marks)	Editor layout is clean and intuitive. Buttons and display are styled clearly.	
Validation and Error Handling (1 Mark)	Stack overflows/empty pop operations are handled gracefully.	
Presentation (5 Marks)	Presentation taken Individually	
Report Submission (5 Marks)	On Date submission and Proper report format	

2. Railway Reservation System Using Linked List

Objective:
Simulate a railway ticket reservation system where each passenger is stored as a **node in a linked list**.

Features:

1. **Passenger Booking**
   • Add new passengers to the list with details like Name, Age, Seat No., Status.
2. **Cancellation System**
   • Remove nodes (passenger details) by seat number.
3. **Waitlist Handling**
   • Maintain waitlisted passengers in a separate list.
4. **Display Function**
   • Show all confirmed and waitlisted passengers.
5. **Linked List Use**
   • Use singly or doubly linked list to add/remove/search passengers.

2) Marks Rubrics:

Categories	Obtain Marks	
Passenger Booking (2 Marks)	New passengers can be added with correct details.	
Cancellation System (2 Marks)	Passengers can be removed accurately by seat number.	
Waitlist Management (2 Marks)	Waitlist is handled as a separate list with correct queueing logic.	
Display Functionality (2 Marks)	All passengers and waitlisted customers are displayed clearly.	
Linked List Implementation (2 Marks)	Nodes are dynamically added/removed using a singly or doubly linked list.	
Presentation (5 Marks)	Presentation taken Individually	
Report Submission (5 Marks)	On Date submission and Proper report format	

3. Music Playlist Manager Using Linked List

Objective:
Create a music playlist app where each song is a **node in a linked list**, supporting navigation and control.

Features:

1. **Add Songs to Playlist**
   • Add new songs dynamically with details like Title, Artist, Duration.
2. **Play Next/Previous**
   • Navigate through songs using next/prev buttons (use doubly or circular linked list).
3. **Remove or Reorder Songs**
   • Delete or move songs within the playlist.
4. **Display Playlist**
   • Show all current songs in order.
5. **Play Simulation**
   • Simulate "Now Playing" with visual/audio cues.

3) Marks Rubrics:

Categories	Obtain Marks	
Add Songs to Playlist (2 Marks)	Songs can be added dynamically with proper details.	
Play Next/Previous (2 Marks)	Navigation through songs works using linked list logic.	
Remove/Reorder Songs (2 Marks)	Songs can be removed or rearranged in the list successfully.	
Display Playlist (2 Marks)	Playlist is displayed in order with song details.	
Simulation and UI (2 Marks)	Now playing indicators or audio simulation is clearly shown.	
Presentation (5 Marks)	Presentation taken Individually	
Report Submission (5 Marks)	On Date submission and Proper report format	

4. Hangman Game Using Data Structures

Objective:
Implement the Hangman word-guessing game using arrays and sets to manage guessed letters and game state.

Features:

1. **Word Selection**
   • Choose a random word from a predefined list.
2. **Guess Input**
   • User inputs one letter at a time.
3. **Tracking Guesses**
   • Use an array/set to store correct and incorrect guesses.
   • Display guessed letters and remaining attempts.
4. **Win/Loss Condition**
   • Display “You Win” or “Game Over” when the game ends.
5. **Interactive UI**
   • Use basic styling and messages to guide the player.

4) Marks Rubrics:

Categories	Obtain Marks	
Word Selection and Setup (2 Marks)	Random word is selected and displayed correctly (masked).	
Guess Input and Validation (2 Marks)	User input is validated, only letters accepted.	
Tracking Guesses (2 Marks)	Correct/incorrect guesses are stored using arrays/sets and updated correctly.	
Game Outcome Logic (2 Marks)	Win/Loss conditions are implemented properly with relevant messages.	
Interactive UI (2 Marks)	Layout is clear and user is guided with dynamic messages or attempts left.	
Presentation(5 Marks)	Presentation taken Individually	
Report Submission (5 Marks)	On Date submission and Proper report format	

5. Restaurant Waitlist Manager Using Queue

Objective:
Manage customer waitlists using **queue data structures** for a restaurant.

Features:

1. **Add Customer to Waitlist**
   • Enqueue customer name, party size, and arrival time.
2. **Serve Next Customer**
   • Dequeue and show who gets the next available table.
3. **View Current Waitlist**
   • Display the queue with all customers in waiting.
4. **Cancel Reservation**
   • Optional: Search and remove a customer if they leave.
5. **Queue Logic**
   • First-come-first-served system using FIFO logic.

5) Marks Rubrics:

Categories	Obtain Marks	
Add to Waitlist (2 Marks)	Customer details (name, size, time) are added correctly to queue.	
Serve Next Customer (2 Marks)	Next in queue is served and dequeued successfully.	
View Current Waitlist (2 Marks)	Queue is displayed in the correct order with all customer info.	
Cancel Reservation (Optional) (2 Marks)	A specific reservation can be removed based on search (bonus functionality).	
Queue Implementation (2 Marks)	Queue uses FIFO logic and is implemented correctly using arrays or linked list.	
Presentation (5 Marks)	Presentation taken Individually	
Report Submission (5 Marks)	On Date submission and Proper report format	

6. Tic Tac Toe Game Using Array

Objective:
Build a simple **Tic Tac Toe** game using a **2D array** to manage game state and logic.

Features:

1. **Grid Layout**
   • Create a 3x3 array grid using HTML/JavaScript.
2. **Player Turns**
   • Alternate between Player X and O.
3. **Win Check**
   • After every move, check rows, columns, diagonals using array.
4. **Game Reset Option**
   • Allow resetting the game at any point.
5. **Visual UI**
   • Show the grid, player indicators, and messages for win/draw.

6) Marks Rubrics:

Categories	Obtain Marks	
Grid and Layout (2 Marks)	3x3 Grid is correctly implemented and styled.	
Player Turn Handling (2 Marks)	Alternates between X and O properly.	
Win Condition Check (2 Marks)	Win/draw is checked correctly using rows/cols/diagonals.	
Reset Option (2 Marks)	Game can be restarted without reloading the page.	
Visual UI and Messages (2 Marks)	Shows whose turn it is and announces result clearly.	
Presentation (5 Marks)	Presentation taken Individually	
Report Submission (5 Marks)	On Date submission and Proper report format	